

Universidad

Arquitectura de Computadoras

Diseño e implementación de una ALU combinacional parametrizable

Trabajo Práctico N.º 1

Estudiante: Javier

Fecha: 1 de septiembre de 2026

Índice

1. Introducción	1
2. Objetivos	1
3. Arquitectura general	1
4. Interfaz y parametrización	2
5. Operaciones implementadas	2
6. Implementación de la lógica combinacional	2
7. Integración con la interfaz de la FPGA	3
8. Ejemplos de funcionamiento	4
9. Verificación mediante simulación	5
10. Consideraciones de síntesis	5
11. Conclusiones	6

Resumen

Este informe presenta el diseño de una unidad aritmético-lógica (ALU) de ocho bits descrita en Verilog y destinada a su implementación en una FPGA. El núcleo de la ALU se construye exclusivamente con lógica combinacional: su salida depende en cada instante de dos operandos y de un código de operación, sin almacenar estado ni utilizar reloj. Se describen la interfaz parametrizable, las operaciones implementadas, la interpretación de datos con signo y la integración con un módulo superior que permite cargar los valores mediante switches y botones. Finalmente, se explica la estrategia de simulación mediante testbench y se presentan ejemplos de cálculo que permiten verificar el funcionamiento esperado.

1. Introducción

Una unidad aritmético-lógica es un bloque fundamental de un procesador: recibe operandos, ejecuta una operación determinada por una señal de control y entrega un resultado. En el presente trabajo se implementa una ALU parametrizable en Verilog, con operandos y resultado de ocho bits y un código de operación de seis bits.

El propósito central no es escribir un algoritmo secuencial, sino describir el hardware que una herramienta de síntesis puede construir en la FPGA. En este contexto, una expresión como

```
o_data = i_data_a + i_data_b;
```

representa la lógica de un sumador. La selección entre varias operaciones se implementa mediante lógica combinacional equivalente a un multiplexor.

2. Objetivos

Los objetivos del trabajo son:

- Diseñar una ALU de ocho bits con operaciones aritméticas, lógicas y de desplazamiento.
- Describir el núcleo funcional utilizando lógica combinacional en Verilog, sin registros ni señales de reloj dentro de la ALU.
- Parametrizar el ancho de datos y de operación para favorecer la reutilización del módulo.
- Integrar el núcleo con una interfaz superior para una placa FPGA, permitiendo cargar y conservar los operandos desde switches.
- Verificar el comportamiento mediante simulación y relacionar las formas de onda con los resultados matemáticos esperados.

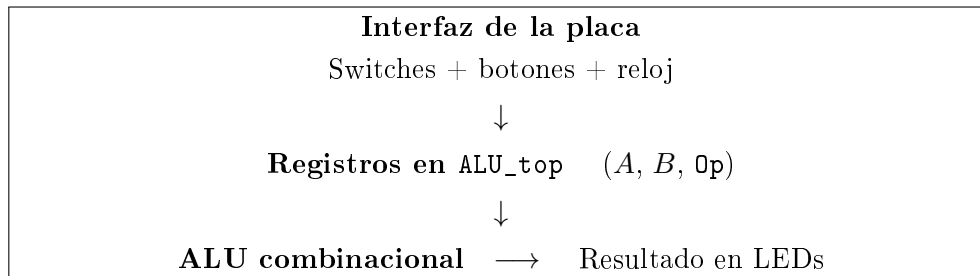
3. Arquitectura general

La solución se divide en dos módulos con responsabilidades diferentes:

`ALU.v` contiene el núcleo combinacional. Recibe los operandos `i_data_a`, `i_data_b` y el código `i_op`; produce `o_data`.

`ALU_top.v` constituye el módulo superior. Conecta la ALU con el reloj, el reset, los botones, el bus de switches y los LEDs. Incluye registros para conservar las entradas cargadas.

La arquitectura puede resumirse de la siguiente manera:



Esta separación es esencial. La ALU propiamente dicha es combinacional; los registros del módulo superior no forman parte de su cálculo, sino que permiten que el usuario cargue A , luego B y finalmente la operación sin perder los valores previamente seleccionados. El resultado no se registra: se actualiza automáticamente cuando cambia alguna de las entradas almacenadas.

4. Interfaz y parametrización

La declaración del módulo permite modificar el tamaño de los buses mediante parámetros:

Listing 1: Interfaz parametrizable del núcleo

```

1 module ALU #(
2     parameter NB_OP = 6,
3     parameter NB_DATA = 8
4 ) (
5     input wire signed [NB_DATA-1:0] i_data_a,
6     input wire signed [NB_DATA-1:0] i_data_b,
7     input wire [NB_OP-1:0] i_op,
8     output reg signed [NB_DATA-1:0] o_data
9 );
  
```

Con la configuración utilizada, A y B son operandos de ocho bits y i_op posee seis bits. Si se instancia el módulo con $NB_DATA = 16$, los operandos y el resultado pasan a ser de dieciséis bits sin modificar la estructura de la ALU. El calificador **signed** permite interpretar los operandos de datos en complemento a dos para las operaciones aritméticas y para el desplazamiento aritmético.

5. Operaciones implementadas

Los códigos de operación se definen mediante `localparam`, lo que mejora la legibilidad del circuito y evita repetir constantes sin nombre en la descripción. La Tabla 1 resume el conjunto implementado.

En las operaciones de desplazamiento, el segundo operando se utiliza como cantidad de desplazamiento. La diferencia entre SRA y SRL se observa cuando el operando A es negativo: SRA replica el bit de signo, mientras que SRL introduce ceros en las posiciones más significativas.

6. Implementación de la lógica combinacional

La lógica del núcleo se describe mediante un bloque `always @(*)`. La lista sensible implícita indica que el bloque debe reevaluarse cuando cambia cualquiera de sus entradas. La asignación inicial garantiza un valor definido para la salida y evita la inferencia accidental de un latch.

Cuadro 1: Códigos y operaciones de la ALU.

Hex.	Binario	Operación	Descripción
20	100000	ADD	$A + B$
22	100010	SUB	$A - B$
24	100100	AND	$A \& B$
25	100101	OR	$A B$
26	100110	XOR	$A \wedge B$
03	000011	SRA	A desplazado aritméticamente a la derecha B posiciones
02	000010	SRL	A desplazado lógicamente a la derecha B posiciones
27	100111	NOR	$\sim(A B)$

Listing 2: Selección combinacional de operaciones

```

1 always @(*) begin
2     o_data = {NB_DATA{1'b0}};
3
4     case (i_op)
5         OP_ADD: o_data = i_data_a + i_data_b;
6         OP_SUB: o_data = i_data_a - i_data_b;
7         OP_AND: o_data = i_data_a & i_data_b;
8         OP_OR:  o_data = i_data_a | i_data_b;
9         OP_XOR: o_data = i_data_a ^ i_data_b;
10        OP_NOR: o_data = ~(i_data_a | i_data_b);
11        OP_SRL: o_data = i_data_a >> i_data_b;
12        OP_SRA: o_data = $signed(i_data_a) >>> i_data_b;
13        default: o_data = {NB_DATA{1'b0}};
14    endcase
15 end

```

La sentencia **case** selecciona la operación asociada con el código de control. Conceptualmente, cada operación alimenta una entrada de un multiplexor y **i_op** determina cuál de ellas llega a la salida. La variable **o_data** se declara como **reg** porque recibe asignaciones dentro de un bloque **always**; esto no significa que se implemente un registro físico. Al no existir un flanco de reloj en este bloque, la síntesis produce lógica combinacional.

Para códigos no contemplados se entrega cero. Además de hacer determinista el comportamiento, esta decisión completa la asignación de la salida para todos los caminos posibles.

7. Integración con la interfaz de la FPGA

El módulo superior utiliza un bus compartido de switches y tres botones de carga. En cada flanco ascendente del reloj se almacena el valor presente en el bus según el botón presionado:

Listing 3: Carga de entradas en el módulo superior

```

1 always @(posedge clock) begin
2     if (reset) begin
3         reg_data_A <= {(DATA_LEN){1'b0}};
4         reg_data_B <= {(DATA_LEN){1'b0}};
5         reg_op      <= {(OP_LEN){1'b0}};
6     end

```

```

7   else if (button_1)
8       reg_data_A <= bus[DATA_LEN-1:0];
9   else if (button_2)
10      reg_data_B <= bus[DATA_LEN-1:0];
11  else if (button_3)
12      reg_op      <= bus[OP_LEN-1:0];
13  end

```

La secuencia de uso es la siguiente:

1. Configurar el valor de A en los switches y presionar el botón 1.
2. Configurar el valor de B y presionar el botón 2.
3. Configurar el código de operación en los seis bits inferiores y presionar el botón 3.
4. Observar el resultado en los LEDs.

La asignación no bloqueante `<=` es apropiada para este bloque secuencial, ya que representa la actualización simultánea de registros en un flanco de reloj. El reset síncrono inicializa los tres registros. Luego, la instancia de la ALU recibe sus valores:

Listing 4: Instanciación del núcleo en el módulo superior

```

1  ALU #(
2      .NB_OP(OP_LEN),
3      .NB_DATA(DATA_LEN)
4  ) alu (
5      .i_data_a(reg_data_A),
6      .i_data_b(reg_data_B),
7      .i_op(reg_op),
8      .o_data(led)
9  );

```

Las conexiones por nombre hacen explícita la correspondencia entre cada puerto y la señal interna, reduciendo el riesgo de errores de conexión.

8. Ejemplos de funcionamiento

Para ilustrar la operación, considérese $A = 5$ y $B = 3$. Las salidas esperadas para algunas operaciones son:

Cuadro 2: Ejemplos de resultados con operandos positivos.

Operación	Código	Cálculo	Resultado
ADD	20	$5 + 3$	8
SUB	22	$5 - 3$	2
AND	24	$00000101 \ \& \ 00000011$	00000001
OR	25	$00000101 \ \ 00000011$	00000111
XOR	26	$00000101 \ \sim \ 00000011$	00000110
NOR	27	$\sim(5 \ \ 3)$	11111000

El resultado de NOR se muestra en ocho bits y equivale a -8 en complemento a dos. Esto evidencia que las operaciones lógicas trabajan sobre el patrón de bits completo, aunque el bus de datos pueda interpretarse con signo.

Como segundo ejemplo, para ADD se pueden seleccionar $A = 2$, $B = \text{FF}$ y el código 20. En ocho bits con signo, FF representa -1 ; por lo tanto,

$$2 + (-1) = 1, \quad (1)$$

y la salida esperada es 01. La aritmética se realiza con el ancho del bus, por lo que los resultados que exceden el rango representable se conservan en los ocho bits menos significativos.

9. Verificación mediante simulación

La verificación funcional se realiza con `tb_ALU.v`, un testbench que instancia directamente el núcleo. Esta elección permite probar la ALU sin incluir botones, reloj ni restricciones físicas de la placa. El estímulo asigna operandos, cambia el código de operación, espera un intervalo de tiempo y observa `o_data`.

En una simulación comportamental se deben visualizar, como mínimo, las siguientes señales:

`i_data_a[7:0]` `i_data_b[7:0]` `i_op[5:0]` `o_data[7:0]`

Para facilitar la lectura, los datos pueden observarse en decimal con signo o hexadecimal, mientras que el código de operación se puede observar en binario o hexadecimal.

El testbench cambia la operación cada 10 ns y conserva el mismo par de operandos durante la secuencia. Los intervalos relativos son:

Cuadro 3: Secuencia temporal de operaciones del testbench.

Intervalo	Operación
0–10 ns	ADD
10–20 ns	SUB
20–30 ns	AND
30–40 ns	OR
40–50 ns	XOR
50–60 ns	SRA
60–70 ns	SRL
70–80 ns	NOR

Para validar una operación se debe ubicar el cursor de la forma de onda en el interior de un intervalo estable, evitando una transición, y comparar el valor de salida con la ecuación correspondiente. Por ejemplo, si se observan `i_data_a = 02`, `i_data_b = FF`, `i_op = 20` y `o_data = 01`, la suma verificada es $2 + (-1) = 1$.

El módulo superior también puede simularse con `tb_ALU_TOP.v`, pero requiere generar correctamente el reloj, el reset, el bus y los botones. Por ello, para verificar primero la función de la ALU se recomienda utilizar `Test_Bench_ALU`; `ALU_top` representa la integración con la placa y no constituye por sí mismo un estímulo de simulación.

10. Consideraciones de síntesis

Desde el punto de vista de hardware, el núcleo combina:

- operadores aritméticos para suma y resta;

- compuertas bit a bit para AND, OR, XOR y NOR;
- lógica de desplazamiento para SRA y SRL;
- lógica de selección para entregar al puerto de salida la operación elegida.

No hay memoria interna en `ALU.v`. El tiempo de respuesta queda determinado por el retardo de propagación de la red combinacional sintetizada. En cambio, los registros de `ALU_top.v` son necesarios para desacoplar la entrada manual de los switches del cálculo: una vez cargados, mantienen los operandos aunque el bus cambie.

El archivo de restricciones `.xdc` completa la implementación física al asociar los puertos del módulo superior con los pines de la placa FPGA. De esta manera, la descripción lógica, la interfaz de usuario y la asignación física quedan separadas en archivos con responsabilidades claras.

11. Conclusiones

Se diseñó una ALU parametrizable de ocho bits capaz de realizar ocho operaciones aritméticas, lógicas y de desplazamiento. La implementación de `ALU.v` cumple el objetivo principal del trabajo: el resultado se obtiene exclusivamente a partir de las entradas actuales y no depende de un reloj ni conserva estado.

La integración mediante `ALU_top.v` mostró por qué una aplicación física necesita registros aun cuando el núcleo sea combinacional. Estos registros almacenan *A*, *B* y *Op*, mientras que la ALU calcula y actualiza el resultado de forma inmediata ante cualquier cambio de dichos valores. La división modular facilita tanto la reutilización del núcleo como su verificación independiente mediante testbench.

Finalmente, la simulación permite relacionar cada código de operación con la ecuación y el patrón de bits observados en la forma de onda. Este procedimiento constituye una verificación funcional previa a la síntesis y a la prueba sobre la placa FPGA.